Deloitte Cyber Gurukul Coding Compendium

Comprehensive Compilation of Previous Year and Frequent Assessment Coding Questions with Python 3 Solutions

1. Corona for Computer (Bitwise Manipulation)

Problem Statement: A binary-eating virus ("Corona for Computer") removes n LSB (right-side) binary digits from each number in an array.

Input: *N (array size), array elements space-separated, and n (number of bits to shift).*

```
N = int(input())
arr = list(map(int, input().split()))
n = int(input())
print(*(x >> n for x in arr))
```

2. Particle Momentum

Problem Statement: Calculate total system momentum (Σ mass × velocity) for n particles traveling in the same direction.

Input: *n (number of particles), followed by n lines containing space-separated mass and velocity values.*

```
n = int(input())
total_momentum = 0
for _ in range(n):
    m, v = map(int, input().split())
    total_momentum += m * v
print(total_momentum)
```

3. Odd Sum Subarrays

Problem Statement: Count the number of contiguous subarrays that sum up to an odd number using prefix sum logic.

```
N = int(input())
arr = list(map(int, input().split()))
odd_c, even_c, res, pref = 0, 1, 0, 0
for num in arr:
    pref += num
    if pref % 2 == 0:
        res += odd_c
        even_c += 1
    else:
        res += even_c
        odd_c += 1
print(res)
```

4. Class Monitor (Rank Dropping Count)

Problem Statement: Count how many times a student's score drops strictly below the running minimum recorded so far.

```
n = int(input())
scores = list(map(int, input().split()))
if not scores:
    print(0)
else:
    curr_min = scores[0]
    drops = 0
    for s in scores[1:]:
        if s < curr_min:
            curr_min = s
            drops += 1
    print(drops)
```

5. Key Generation (Password Hacking)

Problem Statement: A security algorithm requires you to generate a secure passcode from a given string. You must sum the digits of all numeric characters present in the string. If no digits are present, output 0.

```
s = input()
digit_sum = sum(int(char) for char in s if char.isdigit())
print(digit_sum)
```

6. Maximum Product Subarray

Problem Statement: Given an array of integers representing network data packet flows, find the contiguous subarray within the array that has the largest product and return that product.

```
n = int(input())
arr = list(map(int, input().split()))
if not arr:
    print(0)
else:
    max_prod = min_prod = ans = arr[0]
    for x in arr[1:]:
        if x < 0:
            max_prod, min_prod = min_prod, max_prod
        max_prod = max(x, max_prod * x)
        min_prod = min(x, min_prod * x)
        ans = max(ans, max_prod)
    print(ans)
```

7. Distinct Array Elements After Modification

Problem Statement: You are given an array of size N. You can change the sign of any element (multiply by -1) at most once. Find the maximum number of distinct elements you can achieve in the final array.

```
n = int(input())
arr = list(map(int, input().split()))
seen = set()
for x in arr:
    abs_x = abs(x)
    if abs_x not in seen:
        seen.add(abs_x)
    else:
        seen.add(-abs_x)
print(len(seen))
```

8. Encryption Key Redundancy (String Reduction)

Problem Statement: An encryption algorithm reduces string sizes by removing adjacent matching character pairs iteratively until no more pairs can be removed.

```
s = input()
stack = []
for char in s:
    if stack and stack[-1] == char:
        stack.pop()
    else:
        stack.append(char)
ans = "".join(stack)
print(ans if ans else "Empty String")
```

9. Unusual Token Pairs (Target Difference)

Problem Statement: Find the total number of unique pairs of integers (A, B) in an array such that their absolute difference is exactly K (|A - B| = K).

```
from collections import Counter
n = int(input())
arr = list(map(int, input().split()))
k = int(input())
seen = set(arr)
count = 0
if k == 0:
    counts = Counter(arr)
    count = sum(1 for x in counts if counts[x] > 1)
else:
    for x in seen:
        if x + k in seen:
            count += 1
print(count)
```

10. Server Load Balancing (Equilibrium Index)

Problem Statement: Locate an index where the sum of processing weights on the left servers equals the sum of processing weights on the right servers.

```
n = int(input())
weights = list(map(int, input().split()))
total_sum = sum(weights)
left_sum = 0
equilibrium_index = -1
for i, w in enumerate(weights):
    total_sum -= w
    if left_sum == total_sum:
        equilibrium_index = i
        break
    left_sum += w
print(equilibrium_index)
```

11. Bitwise Transmission Parity

Problem Statement: Find the element in an array that appears an odd number of times, while all other data packets/elements appear exactly an even number of times.

```
n = int(input())
packets = list(map(int, input().split()))
res = 0
for p in packets:
    res ^= p
print(res)
```

12. Prime Sum with Ending Digit 'K'

Problem Statement: Find the sum of the first N prime numbers that end with a specific digit K.

```
def is_prime(num):
    if num < 2: return False
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0: return False
    return True

N, K = map(int, input().split())
primes_found, current_num, total_sum = 0, 2, 0
while primes_found < N:
    if current_num % 10 == K and is_prime(current_num):
        total_sum += current_num
        primes_found += 1
    current_num += 1
print(total_sum)
```

13. Reference String Character Filter

Problem Statement: Given a target string and a reference string, remove all characters from the target string that appear anywhere in the reference string.

```
target = input()
reference = set(input())
print("".join([char for char in target if char not in reference]))
```

14. Dynamic String Space-Pairing

Problem Statement: Modify a string by inserting a space after every 2 characters. If the trailing character is left alone without a pair, print it by itself.

```
s = input().replace(" ", "")
print(" ".join(s[i:i+2] for i in range(0, len(s), 2)))
```